

Benchmarking

Laboratorio di Intelligenza Artificiale

Vincenzo Bonnici

Corso di Laurea Magistrale in Scienze Informatiche

Dipartimento di Scienze Matematiche, Fisiche e Informatiche

Università degli Studi di Parma

2026-2027

- **Benchmarking** (in italiano **analisi comparativa**): "benchmark something (against something) to judge the quality of something in relation to that of other similar things"
 - "The journal was recently benchmarked against other IT journals."
 - "Projects are assessed and benchmarked against the targets."
- **Benchmark**: "something which can be measured and used as a standard that other things can be compared"
 - è costituito da uno o più data set
 - **reali**: acquisiti tramite misurazione di eventi realmente accaduti. Hanno il vantaggio di rappresentare in modo altamente veritiero (a scanso di errori di misurazione) il mondo reale
 - **sintetici** (o *in silico*): costruiti artificialmente seguendo delle regole specifiche. Posso basarsi su dati reali, tuttavia, possono risultare molto distanti dalla realtà.

Nonostante l'avvento di benchmark standardizzati abbia semplificato il processo di confronto delle prestazioni tra diversi sistemi informatici, architetti e ricercatori affrontano diverse sfide quando utilizzano i benchmark per lo sviluppo industriale del prodotto e la ricerca accademica:

- i benchmark rappresentano solo un campione dello spazio "comportamentale" (dei fenomeni) della specifica applicazione
- i benchmark sono rigidi e misurano le prestazioni per un singolo punto di tale spazio
- le suite di benchmark sono costose da sviluppare, mantenere e aggiornare:
 - "Designing tomorrow's microprocessors using today's benchmarks built from yesterday's programs" [WEIC97] [YI06].
- i benchmark standardizzati sono open-source anche se a volte le applicazioni di interesse sono proprietarie

Caratteristiche chiave di un benchmark:

- Rilevanza: Quanto strettamente il comportamento del benchmark corrisponde a comportamenti che interessano i consumatori dei risultati
- Riproducibilità: La capacità di produrre costantemente risultati simili quando il benchmark viene eseguito con la stessa configurazione di test
- Correttezza (fairness): Consentire a diverse configurazioni di test di competere sui loro meriti senza limitazioni artificiali
- Verificabilità: fornendo sicurezze sul fatto che i risultati del benchmark sono accurati
- Usabilità: permettere a tutti gli utenti di eseguire il benchmark nei loro ambienti

REVIEW

Open Access

Essential guidelines for computational method benchmarking



Lukas M. Weber^{1,2}, Wouter Saelens^{3,4}, Robrecht Cannoodt^{3,4}, Charlotte Soneson^{1,2,8}, Alexander Hapfelmeier⁵, Paul P. Gardner⁶, Anne-Laure Boulesteix⁷, Yvan Saeys^{3,4*} and Mark D. Robinson^{1,2*} 

Abstract

In computational biology and other sciences, researchers are frequently faced with a choice between several computational methods for performing data analyses. Benchmarking studies aim to rigorously compare the performance of different methods using well-characterized benchmark datasets, to determine the strengths of each method or to provide recommendations regarding suitable choices of methods for an analysis. However, benchmarking studies must be carefully designed and implemented to provide accurate, unbiased, and informative results. Here, we summarize key practical guidelines and recommendations for performing high-quality benchmarking analyses, based on our experiences in computational biology.

Summary of guidelines

The guidelines in this review can be summarized in the following set of recommendations. Each recommendation is discussed in more detail in the corresponding section in the text.

1. Define the purpose and scope of the benchmark.
2. Include all relevant methods.
3. Select (or design) representative datasets.
4. Choose appropriate parameter values and software versions.
5. Evaluate methods according to key quantitative performance metrics.
6. Evaluate secondary measures including computational requirements, user-friendliness, installation procedures, and documentation quality.
7. Interpret results and provide recommendations from both user and method developer perspectives.
8. Publish results in an accessible format.
9. Design the benchmark to enable future extensions.
10. Follow reproducible research best practices, by making code and data publicly available.

Linee guida per la creazione di benchmark

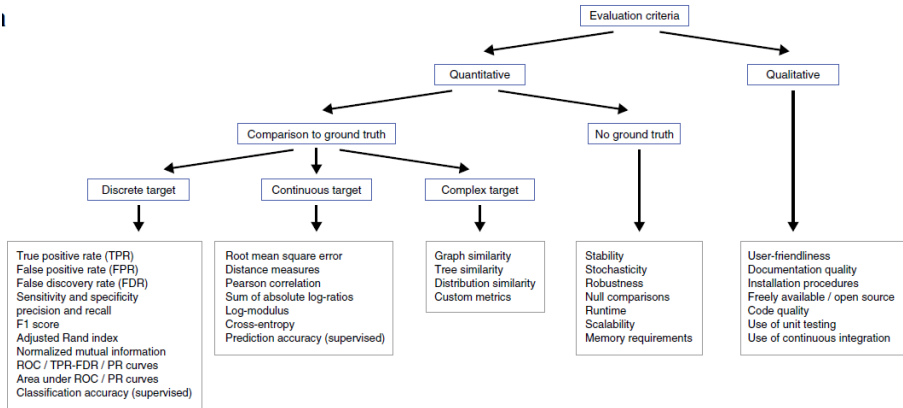
Table 1 Summary of our views regarding 'how essential' each principle is for a truly excellent benchmark, along with examples of key tradeoffs and potential pitfalls relating to each principle

Principle (see Fig. 1)	How essential? ^a	Tradeoffs	Potential pitfalls
1. Defining the purpose and scope	+++	How comprehensive the benchmark should be	Scope too broad: too much work given available resources Scope too narrow: unrepresentative and possibly misleading results
2. Selection of methods	+++	Number of methods to include	Excluding key methods
3. Selection (or design) of datasets	+++	Number and types of datasets to include	Subjectivity in the choice of datasets: e.g., selecting datasets that are unrepresentative of real-world applications Too few datasets or simulation scenarios Overly simplistic simulations
4. Parameter and software versions	++	Amount of parameter tuning	Extensive parameter tuning for some methods while using default parameters for others (e.g., competing methods)
5. Evaluation criteria: key quantitative performance metrics	+++	Number and types of performance metrics	Subjectivity in the choice of metrics: e.g., selecting metrics that do not translate to real-world performance Metrics that give over-optimistic estimates of performance Methods may not be directly comparable according to individual metrics (e.g., if methods are designed for different tasks)
6. Evaluation criteria: secondary measures	++	Number and types of performance metrics	Subjectivity of qualitative measures such as user-friendliness, installation procedures, and documentation quality Subjectivity in relative weighting between multiple metrics Measures such as runtime and scalability depend on processor speed and memory
7. Interpretation, guidelines, and recommendations	++	Generality versus specificity of recommendations	Performance differences between top-ranked methods may be minor Different readers may be interested in different aspects of performance
8. Publication and reporting of results	+	Amount of resources to dedicate to building online resources	Online resources may not be accessible (or may no longer run) several years later
9. Enabling future extensions	++	Amount of resources to dedicate to ensuring extensibility	Selection of methods or datasets for future extensions may be unrepresentative (e.g., due to requests from method authors)
10. Reproducible research best practices	++	Amount of resources to dedicate to reproducibility	Some tools may not be compatible or accessible several years later

^aThe higher the number of plus signs, the more central the principle is to the evaluation

Scelta dei criteri per la valutazione delle prestazioni

1



Algoritmo di ricerca in profondità randomizzata

- si parte da una scacchiera (grafo), in cui ogni cella ha inizialmente 4 pareti attorno ad essa.
- A partire da una cella casuale, si seleziona una cella adiacente casuale che non è stata ancora visitata.
- si rimuove il muro tra le due celle e contrassegna la nuova cella come visitata, e la si aggiunge alla pila della DFS
- si continua il processo. Una cella che non ha vicini non visitati viene considerata un vicolo cieco. Quando si trova in un vicolo cieco, fa un passo indietro attraverso il percorso fino a raggiungere una cella con un vicino non visitato
- si continua la generazione del percorso visitando questa nuova cella non visitata (creando un nuovo incrocio).
- si continua fino a che ogni cella non è stata visitata

Algoritmi per la generazione di labirinti

Algoritmo di ricerca in profondità: versione ricorsiva

- Data una cella corrente come parametro
- Contrassegna la cella corrente come visitata
- Mentre la cella corrente ha celle vicine non visitate
 - Scegli uno dei vicini non visitati
 - Rimuovi il muro tra la cella corrente e la cella scelta
 - Richiama la routine ricorsivamente per una cella scelta

Algoritmo di ricerca in profondità: versione iterativa

- Scegli la cella iniziale, contrassegna come visitata e metterla nello stack
- Fino a che lo stack non è vuoto
 - Estrai una cella dallo stack e rendila la cella corrente
 - Se la cella corrente ha dei vicini che non sono stati visitati
 - Mettere la cella corrente nello stack
 - Scegli uno dei vicini non visitati
 - Rimuovi il muro tra la cella corrente e la cella scelta
 - Contrassegna la cella scelta come visitata e spingila nello stack

Algoritmo randomizzato di Kruskal

- Crea un elenco di tutte le pareti e crea un set per ogni cella, ognuna contenente solo quella cella.
- Per ogni muro, in un ordine casuale:
 - Se le celle divise da questo muro appartengono a insiemi distinti:
 - Rimuovi il muro corrente.
 - Unisciti ai set delle celle precedentemente divise.

Algoritmo randomizzato di Prim

- Inizia con una griglia piena di muri.
- Scegli una cella, contrassegnala come parte del labirinto. Aggiungi i muri della cella all'elenco dei muri.
- Mentre ci sono muri nell'elenco:
 - Scegli un muro casuale dall'elenco.
 - Se viene visitata solo una delle due celle che divide il muro, allora:
 - Trasforma il muro in un passaggio e segna la cella non visitata come parte del labirinto.
 - Aggiungi i muri vicini della cella all'elenco dei muri.
 - Rimuovere il muro dall'elenco

Algoritmi per la generazione di labirinti

Algoritmo randomizzato di Wilson: genera un campione imparziale utilizzando random walk di tipo loop-erase con una distribuzione di probabilità uniforme (serve a non generare troppi vicoli ciechi).
E' indipendente dall'ordine con cui visitiamo le celle ancora non visitate.

La tecnica del loop-erase random walk viene utilizzata per estrarre degli alberi (spanning tree) in modo casuale da un grafo.

Si fa partire una random walk da un vertice del grafo e si estrae quindi un determinato percorso che può contenere dei cicli.

Da tale percorso si eliminano i cicli, trasformando così il percorso in un albero.

Nel caso del labirinto si può decidere se

- lanciare un singolo random walk e attendere che questo visiti tutto il grafo, quindi, eliminare i cicli
- lanciare più random walk tale che il random walk al passo successivo non visiti le celle di random walk dei passi precedenti, quindi, trasformare ogni percorso in un albero e connettere gli alberi

Algoritmo di divisione ricorsiva

- si inizia con un labirinto senza pareti, detto anche camera
- si divide la camera in due con una parete posizionata casualmente tale che la parete ha un buco per passare da una parte all'altra della camera appena separata
- si procede in modo ricorsivo sulle due nuove camere

Automi cellulari

- simile al Game of life di Conway
- una cella sopravvive da una generazione all'altra se e solo se ha tra gli 1 e i 5 (o 4) vicini

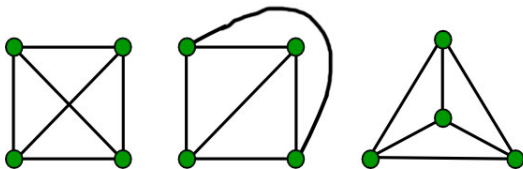
Criteri per valutare la formazione di un "buon" labirinto

- rapporto tra numero di celle di cammino e muri
- numero di vicoli ciechi
- numero di percorsi distinti che portano all'uscita
- lunghezze dei (sotto)percorsi senza diramazioni
- distanza (del percorso minimo) dal punto di entrata al punto di uscita
- ...

Un **grafo planare** è un grafo tale che esso può essere raffigurato in un piano in modo che non si abbiano archi che si intersecano.

In linea generale non è possibile dimostrare in tempo lineare se un grafo è planare o meno. Tuttavia, per un grafo semplice, connesso e planare con n nodi ed e archi si dimostra che:

- Se $n \geq 3$, allora $e \leq 3n - 6$
- Se $n > 3$ e non vi sono cicli di lunghezza 3, allora $e \leq 2n - 4$



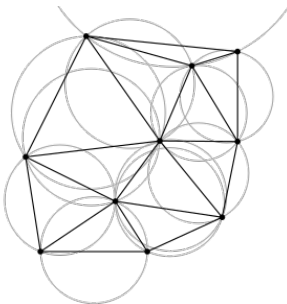
(Il primo grafi a sinistra non è planare).

Algoritmi per la generazione di grafi planari

Esistono varie tecniche per la generazione di grafi planari con particolari proprietà.

Per esempio, la triangolazione di Delaunay per un gruppo di punti P su un piano è una triangolazione $DT(P)$ tale che nessun punto appartenente a P sia all'interno del circumcerchio di ogni triangolo in $DT(P)$.

La triangolazione di Delaunay massimizza il minor angolo di tutti gli angoli dei triangoli nella triangolazione, ovvero, si tende a evitare i triangoli stretti.



Algoritmi per la generazione di grafi planari

Molti di questi algoritmi evitano di generare i grafi da zero e cercano di sfruttare dei grafi già esistenti aggiungendo o rimuovendo vertici o archi. Uno di questi è *Chauhan et al. Applied Network Science (2019)*

